

FTRL-Based Bandit Sampling for Adam Optimization in Deep Learning

System Science Course
Graduate School of Informatic, Kyoto University
劉 晃伸 LIU HUANGSHEN

Supervisor: Prof. Junya Honda

2026.2.3

Background

- Deep learning trains by repeated mini-batch updates (SGD / Adam)
- Uniform sampling will be usually used in standard training
- Samples are not equally informative:
 - large loss / gradients
 - rare or difficult
 - stage-dependent importance
- A viewpoint: focus computation on informative samples to improve wall-clock efficiency.

Preliminaries—Stochastic Optimization

Empirical Risk Minimization

$$\min_{\theta \in \mathbb{R}^d} F(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta)$$

- n : numbers of samples, $f_i(\theta)$: loss of sample i
 - $\theta \in \mathbb{R}^d$: parameters, $F(\theta)$: average empirical risk
-
- The objective is an average over all samples (ERM).
 - Computing full gradients is costly $\rightarrow$ use mini-batches.
 - A mini-batch is a subset sample used to approximate the ERM update (SGD / Adam).

Preliminaries—— Select the sample

Dataset $D = \{(x_i, y_i)\}_{i=1}^n$

Maintain sampling distribution:

$$p_t = (p_{1,t}, \dots, p_{n,t})$$

- Sample a mini-batch $J_t \subset [n]$, with $|J_t| = K$ (without replacement).
- Observe gradients only for $i \in J_t$, and update p_{t+1} accordingly.
- This repeated selection process — with only partial feedback
 $\Rightarrow$ very similar to a **bandit setting**

Preliminaries—— Multi-Armed Bandit(MAB)

Multi-Armed Bandit (MAB)

- A sequential decision problem where, at each round $t = 1, \dots, T$, the learner chooses an arm $a_t \in [n]$ and observes feedback only for the chosen arm.
- learn an arm-selection policy that updates its selection probabilities over time to minimize regret.
- **Connection:** each example (x_i, y_i) is an **arm**; we learn only from the **picked arms** partial feedback (grandient).
 - When we want to select a **Mini-batch** :
pick $K > 1$ **arms** per round
 $\Rightarrow$ **combinatorial semi-bandit**.

Combinatorial adversarial semi-bandit (multiple-play)

At each round $t = 1, \dots, T$, the learner selects a subset $S_t \subset [n]$ with $|S_t| = K$. An adversary assigns item losses $\ell_{t,i} \in [0, 1]$ for all $i \in [n]$, and the learner observes *semi-bandit* feedback $\{\ell_{t,i}\}_{i \in S_t}$ only.

- **Connection to stochastic Optimization:**
 - treat sampling as an **online decision process**
 - update sampling probabilities over time using only **the observed feedback**
- **Practical issue:** Without observing feedback for unselected samples, need to make the feedback **unbiased**
 $\Rightarrow$ use the **importance-weighted estimator**

Preliminaries— Unbiased Loss Estimator

- **Partial feedback:**

in each round we observe the loss only for the **selected** arm (sample)

- **Why bias happens:**

if we simply set the losses of **unselected** arms to 0, the resulting estimate systematically **underestimates** the true loss

Importance-Weighted (IW) Estimator

$$\hat{\ell}_{t,i} := \begin{cases} \frac{\ell_{t,i}}{p_{t,i}}, & \text{if arm } i \text{ is selected,} \\ 0, & \text{if arm } i \text{ is not selected.} \end{cases}$$

Unbiasedness:

$$\mathbb{E}[\hat{\ell}_{t,i}] = \ell_{t,i}.$$

Previous work

- AdamBS (Liu R+2020):
 - Adapts sampling probabilities via EXP3/ Multiplicative Weights (MW)-style updates from semi-bandit feedback.
- AdamCB (Kim+2025):
 - Models mini-batch selection as a combinatorial semi-bandit with a more principled pipeline.
- Common: bandit-style updates for sampling probabilities with fixed sampler learning rate.

$$p_{t+1,i} \propto p_{t,i} \exp\left(-\eta \hat{\ell}_{t,i}\right), \quad i = 1, \dots, n.$$

- Limitation: updates can be **sensitive** under noisy importance-weighted estimates

Proposed Method — Overall

- **Motivation:**

fixed-step multiplicative updates may overreact and cause abrupt probability jumps.

- **Method:**

update the sampling distribution using an **FTRL viewpoint**

compute p_{t+1} via a regularized optimization objective, and keep the resulting exponential-weights structure for efficient updates in practice.

- **Key distinction:**

EXP3/MW is an update rule; **FTRL** derives the update from an optimization principle.

Proposed Method : Motivation

Standard Follow-The-Regularized-Leader (FTRL)

A policy choosing arm according to the arm-selection probability $p \in \Delta^n = \left\{ p \in \mathbb{R}_{\geq 0}^n : \sum_{i=1}^n p_i = 1 \right\}$ given by:

$$p_{t+1} = \arg \min_{p \in \Delta^n} \langle p, Z_t \rangle + \frac{1}{\eta} R(p)$$

- $Z_t \in \mathbb{R}^n$ cumulative score (estimated loss) vector up to time t , $Z_{i,t} = \sum_{s=1}^t \hat{\ell}_{i,s}$.
- $\eta > 0$ learning-rate controlling how strongly we react to Z_t .
- $R(p)$: regularizer that prevents over-confident distributions

- **Online:** update p_t using **IW feedback** $\hat{\ell}_{t,i}$ from partial observations.
- **Stable:** $R(p)$ avoids overly-peaked distributions.

Proposed Method : Variance-Adaptive FTRL

- **Our contribution:**

keep the FTRL-derived exponential form but make it **variance-adaptive** under importance-weighted feedback.

We track per-arm feedback variability and damp overly aggressive updates.

$$S_{i,t} = \sum_{s \leq t} \hat{\ell}_{i,s}^2,$$
$$\eta_{i,t} = \frac{\alpha}{\sqrt{S_{i,t} + \varepsilon}}.$$

- With negative-entropy $R(p) = \sum_i p_i \log p_i$, FTRL admits a closed-form update:

$$p_{i,t+1} = \frac{\exp(-\eta_{i,t} Z_{i,t})}{\sum_{j=1}^n \exp(-\eta_{j,t} Z_{j,t})}.$$

Proposed Method—Feedback Design

Surrogate Feedback Design

- Need a **bounded loss-like** surrogate $\ell_{i,t}$, observable only for $i \in J_t$.
- **FTRL-grad:** $\ell_{i,t}^{\text{grad}} = \psi(\|\nabla f_i(\theta_t)\|)$
 ψ is a monotone scaling (e.g., min–max to $[0, 1]$) that controls the magnitude, so larger gradients yield larger feedback.
- **FTRL-loss:** $\ell_{i,t}^{\text{loss}} = f_i(\theta_t)$
This reuses the task loss computed during training, giving an objective-aligned signal with negligible additional cost.

A Metric to Monitor Probability Changes

Smoothness of Sampling Distribution

Define per-round worst-case change:

$$\text{Smooth}(t) = \|\log p_{t+1} - \log p_t\|_{\infty} = \max_i |\log p_{i,t+1} - \log p_{i,t}|.$$

- Interpretable as the **largest multiplicative change** of probabilities (max-divergence view).
- Smaller $\text{Smooth}(t) \Rightarrow$ fewer abrupt reweighting shocks $\Rightarrow$ potentially more stable training.

- Base distribution (maintained): $p_t \in \Delta^n := \{p \in \mathbb{R}_+^n : \sum_{i=1}^n p_i = 1\}$
Choose a mini-batch $J_t \subset [n]$ with $|J_t| = K$

- Sampling (w/o replacement): convert p_t to **inclusion probabilities**

$$\pi_t \in [0, 1]^n, \quad \sum_{i=1}^n \pi_{i,t} = K, \quad \Pr(i \in J_t) = \pi_{i,t},$$

then sample J_t via DepRound(π_t) (Gandhi+2006)

- Semi-bandit IW feedback (use $\pi_{i,t}$ since it is the true inclusion prob):

$$\hat{\ell}_{i,t} = \begin{cases} \ell_{i,t} / \pi_{i,t}, & i \in J_t, \\ 0, & i \notin J_t, \end{cases}$$

Proposed Method—Algorithm

Algorithm 1: FTRL-based adaptive mini-batch sampling (overview)

Input: batch size K ; parameters α, ε

Init: p_1 (e.g., uniform); $Z_{i,0} = 0$, $S_{i,0} = 0$ for all i

for $t = 1, 2, \dots$ **do**

(1) Sampling:

 construct inclusion probabilities π_t from p_t with $\sum_i \pi_{i,t} = K$; sample mini-batch J_t (w/o repl.) via $\text{DEPROUND}(\pi_t)$;

(2) Training:

 update the model (e.g., Adam) on $i \in J_t$ using unbiased importance-weighted gradients;

(3) Probability update (FTRL):

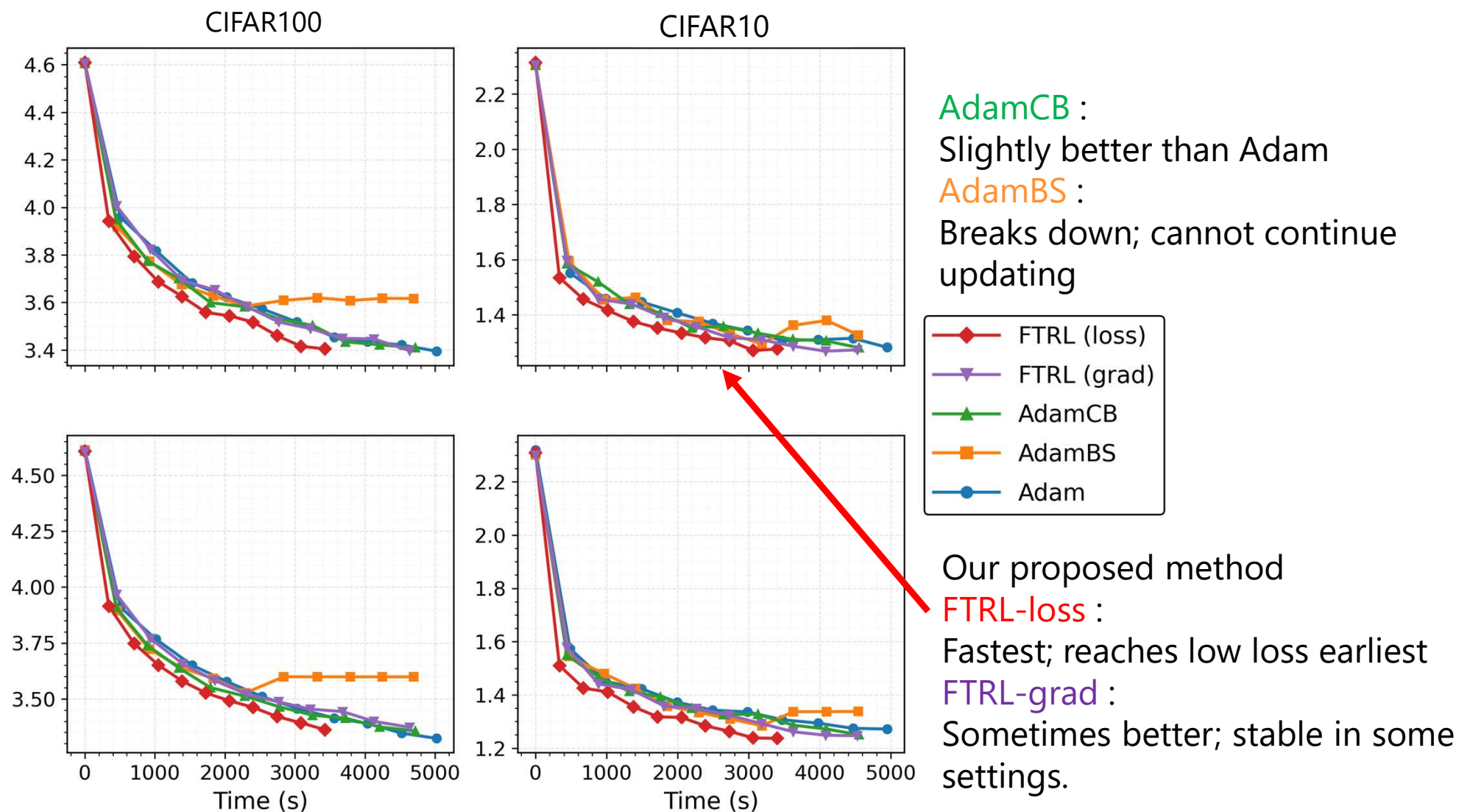
 from the observed feedback on J_t , update (Z_t, S_t) , set per-arm step size η_t , and obtain p_{t+1} (details in the previous slides);

- **Datasets / tasks:** MNIST, Fashion-MNIST, CIFAR-10/100 (image classification).
- **Models:** CNN (MNIST/Fashion/CIFAR) + LSTM (MNIST/Fashion) for a sequence benchmark.
- **Methods:** Uniform Adam; AdamBS (NeurIPS'20); AdamCB (ICLR'25); **Ours** (FTRL-loss / FTRL-grad).
- **Evaluation:** test loss/accuracy **vs. wall-clock time** + sampling stability $\text{Smooth}(t)$.

Experiments 1 — CNN results: training loss and test loss

- FTRL-based samplers (loss/grad) reach lower loss earlier and show smoother convergence than Adam and bandit baselines, with the clearest gain on CIFAR-10/100.

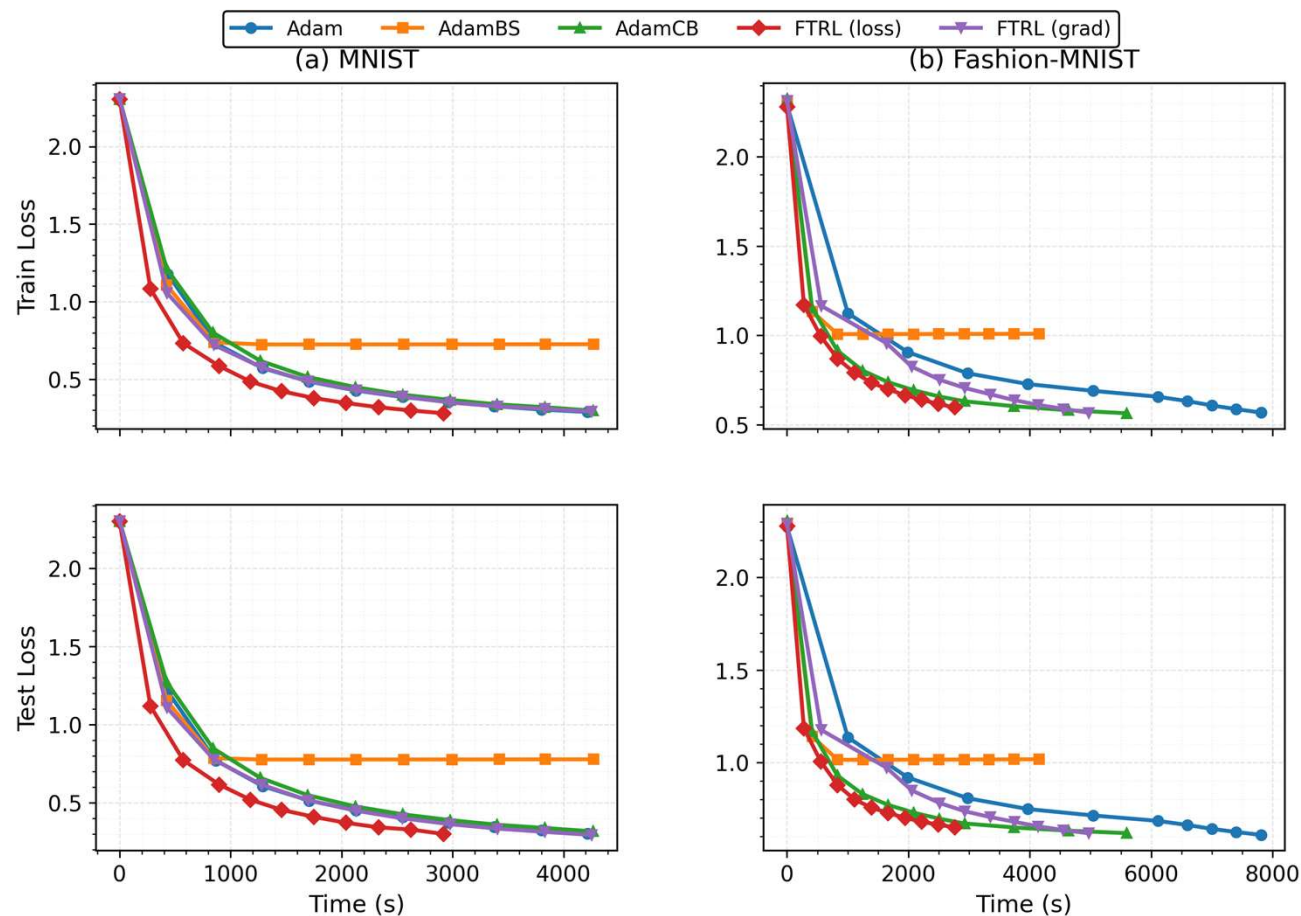
The loss curve: **Lower and earlier loss is better** ✓



Experiments 2 — Image LSTM results

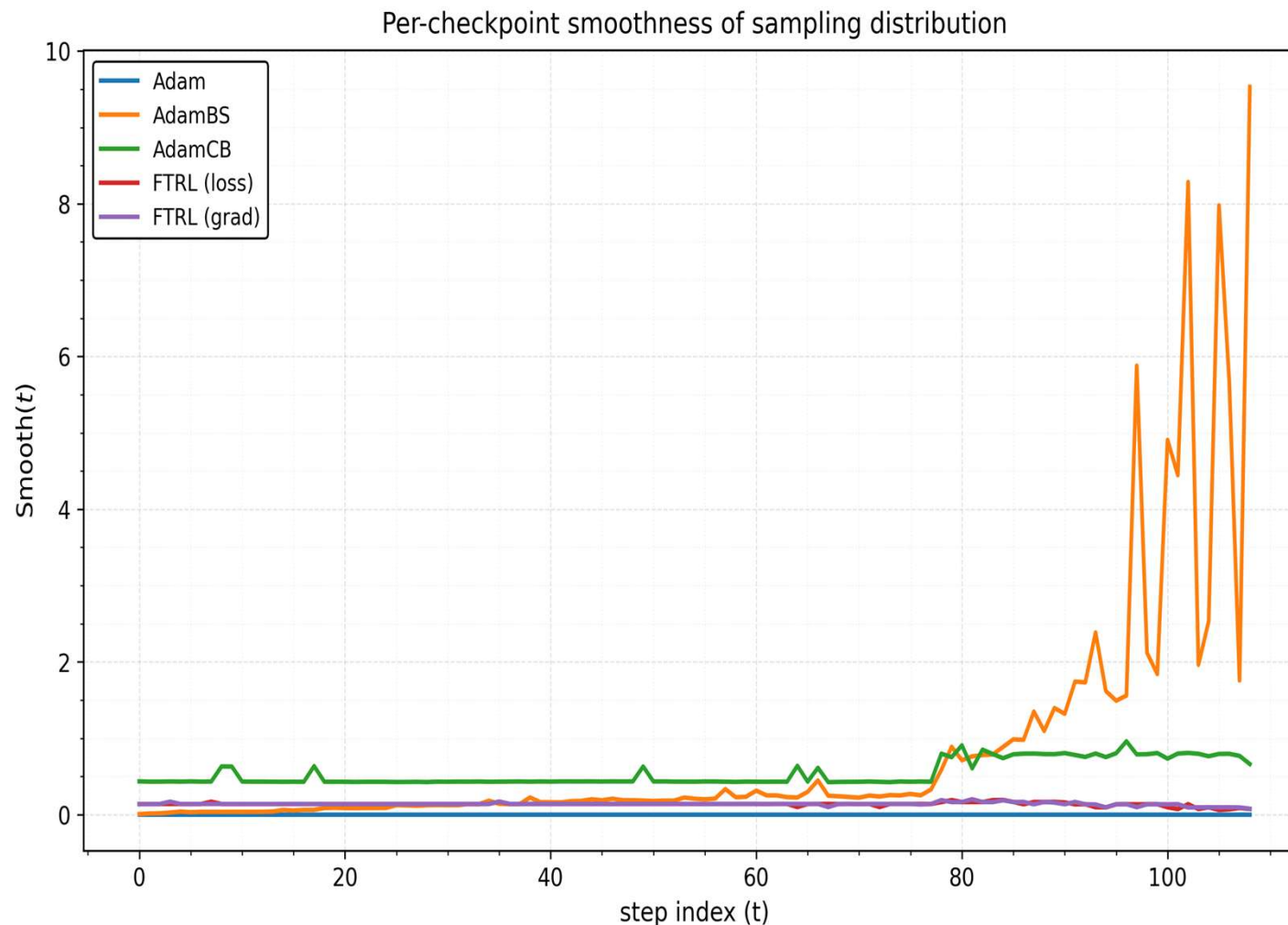
- FTRL-loss achieves lower loss earlier than Adam/AdamBS on both datasets.
- FTRL-grad shows similar performance to AdamCB under this setting

RNN Results (2×2): Train/Test Loss



Experiments 3 — smoothness of sampling distribution¹⁹

- FTRL-based samplers keep the sampling distribution consistently smoother over time
- AdamBS shows late-stage spikes (abrupt probability shifts)
- AdamCB is more stable than AdamBS but still less smooth than FTRL



Conclusion

- Formulated adaptive mini-batch selection as a combinatorial semi-bandit.
- Proposed an FTRL-based probability update for stable sampling under importance weight feedback.
- Empirically, achieves competitive wall-clock efficiency with smoother probability dynamics.

Future Work

- Tighter analysis of the stability–performance tradeoff (theory + diagnostics).
- Extend to larger-scale tasks and broader model families.
- Explore alternative feedback designs and probability updates.